

NAME:

Practice Problems for Week 4

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Wednesday 4/29/26**.

You will be submitting the following files:

- `sum_of_powers.py`
- `primes.py`

Programming Part 1: Sums of Powers

Sums of powers arise in many contexts in maths. A sum of powers is a sequence of the first n integers raised to a given power. For example, the sum of the first 5 squares is: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$

1. Download the starter file `sum_of_powers.py` from the course website.
2. Write a function called `sum_of_powers` which, given an exponent and an integer n that indicates how many numbers to sum up, calculates (and returns) the first n integers raised to the given exponent.
3. Save your program in a file called `sum_of_powers.py`.
4. Select a few inputs for your function and calculate what the correct output of your function should be for those inputs by hand (or with a calculator). Use these inputs to test your function.

Programming Part 2: Primes

A prime number is a natural number greater than 1 that is only evenly divisible by itself and 1. In this assignment you will define a function called `is_prime` that checks whether a given number is a prime number.

1. Create a file called `primes.py`, which you'll use for this assignment.
2. Before writing any code, answer the following questions. Add the answers as comments to the top of your file.
 - a. How many parameters does this function need and what do they represent?
 - b. Should this function have a return value? If so, what kind of value will be returned? (e.g. number, string, boolean?)
 - c. Give a few example function calls that you could use to test whether your implementation works correctly.
3. The name of the function you create should be `is_prime`. Below is an algorithm you can use for your function. Note: You only have to worry about positive integers.
 - a. Assume that x is the number that you're checking to determine if it's prime.

- b. If x is smaller than 2, it is not a prime number, so you can return False immediately.
 - c. Otherwise, define a boolean variable, which represents whether or not x is prime. Initialize this variable to (i.e. assign it the value) True. Note: in effect, we begin by assuming that x is prime.
 - d. For all numbers between 2 and x , check whether x is evenly divisible by the number. If so, x is not prime. Record this by changing the value of the boolean variable you defined in the previous step.
 - e. At the end, the value of the boolean variable should reflect whether or not x is prime, so all you need to do is return that value.
4. (Optional) Can you improve on this algorithm to make it more efficient? Hint 1: Aside from x itself, what is the largest number x can possibly be evenly divisible by? Hint 2: Are there any numbers between 2 and x that you don't need to check?

Written Questions

1. Which of the following are expressions? Which of the expressions are Boolean expressions (ie. conditions)? What value is described by each of the Boolean expressions? Assume that the following variables have been defined:

$x = 7$
 $y = 2$

Consider the following problem specification: Define a function that doubles a numeric (int or float) parameter. Which of the following successfully implement this function and demonstrate to a user that it works? For those that don't satisfy both requirements, explain what the problem(s) is/are.

1.

```
def double(a):  
    result = 2*a  
    return result  
  
print(result)
```

.....
.....
.....
.....
.....
2.

```
def double(a):  
    result = 2*a  
    return result  
  
double(10)  
print(result)
```

.....
.....
.....
.....
.....
3.

```
def double(a):  
    result = 2*a  
    return result  
  
double(10)  
print(return)
```

.....
.....
.....
.....
.....

```
4. def double(a):  
    result = 2*a  
    return result  
  
x = double(10)  
print(x)
```

```
.....  
.....  
.....  
.....  
.....
```

```
5. def double(a):  
    result = 2*a  
    return result  
  
x = double(10)  
print(double(10))
```

```
.....  
.....  
.....  
.....  
.....
```

```
6. def double(a):  
    result = 2*a  
    return result  
  
print(double(10))
```

```
.....  
.....  
.....  
.....  
.....
```

```
7. def double(a):  
    result = 2*a  
    return result  
  
x = double(10)  
print(double(10))
```

```
.....  
.....  
.....  
.....  
.....
```

```
8. def double(a):  
    result = 2*a  
    print result  
  
print(double(10))
```

```
.....  
.....  
.....  
.....  
.....
```

```
9. def double(a):  
    result = 2*a  
    print result  
  
double(10)
```

```
.....  
.....  
.....  
.....  
.....
```